Predictive Maintenance Advisor for Industrial Equipment

An AI-Driven Condition-Based Monitoring System for SME Manufacturing

Team APEX

Ashwin S, Nagulan M, Hemchudesh A, Surya C

Department of Information Technology

K. Ramakrishnan College of Technology, Tiruchirappalli, Tamil Nadu, India

TENSOR '26 Hackathon — Manufacturing & Industry Track (PS07)

Abstract

Unplanned equipment downtime is a persistent and costly problem for small and medium-sized manufacturing enterprises (SMEs), where time-based preventive maintenance remains the dominant strategy despite its well-documented inefficiencies. Commercial predictive maintenance (PdM) platforms offer condition-based alternatives but are priced far beyond the reach of most SMEs. This paper presents APEX, an AI-assisted predictive maintenance advisor that ingests multivariate sensor telemetry from industrial equipment, forecasts Remaining Useful Life (RUL) with calibrated uncertainty, and generates ranked, plain-language maintenance recommendations on a real-time factory floor dashboard. The system combines a one-dimensional Convolutional Neural Network with a Bidirectional Long Short-Term Memory network (CNN-BiLSTM), augmented with Monte Carlo Dropout to quantify prediction uncertainty. Trained on the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) FD001 benchmark, the model achieves an RMSE of 14.2 cycles and an F1-score of 0.89 for a 30-cycle failure-horizon classification task, outperforming an XGBoost baseline (RMSE 19.8, F1 0.74). End-to-end dashboard update latency is measured at 0.4 seconds (p50) and 1.1 seconds (p99), comfortably within the 3-second real-time requirement. APEX demonstrates that cost-effective, deployable predictive maintenance is achievable for SMEs using open-source tooling and publicly available benchmark data.

***Keywords* —** Predictive Maintenance, Remaining Useful Life, CNN-BiLSTM, Monte Carlo Dropout, NASA C-MAPSS, Industrial IoT, Condition-Based Monitoring, Real-Time Dashboard.

1. Problem Statement and Objective

1.1 Problem Statement

Unplanned equipment downtime in manufacturing facilities is estimated to cost the global industry billions of dollars annually. Despite the maturity of Industrial Internet of Things (IIoT) sensing, most SME manufacturers continue to rely on fixed calendar-based maintenance schedules — replacing components at predetermined intervals regardless of actual wear. This strategy simultaneously wastes the useful life of healthy parts and fails to anticipate genuine imminent failures, leading to both inflated maintenance costs and disruptive unplanned shutdowns.

While enterprise-grade predictive maintenance platforms such as Siemens MindSphere and GE Predix offer sophisticated condition-based monitoring, their licensing costs, integration complexity, and infrastructure requirements place them out of reach of the vast majority of small and medium manufacturers. There is a

clear and pressing need for an affordable, deployable, AI-driven predictive maintenance solution tailored to the operational and financial realities of SMEs.

1.2 Objective

The objective of the APEX project is to design and implement an end-to-end predictive maintenance advisory system that:

1. Ingests multivariate time-series sensor telemetry from industrial equipment in near real time.
2. Predicts the Remaining Useful Life (RUL) of each monitored machine using a deep learning model with calibrated uncertainty estimates.
3. Translates raw model outputs into ranked, plain-language maintenance advisories that a shop-floor supervisor can act on without specialized data science knowledge.
4. Presents all of the above on an interactive, low-latency web dashboard suitable for deployment on a standard factory-floor workstation.

2. Methodology and Approach

2.1 Dataset

The system is trained and evaluated on the publicly available NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) turbofan engine degradation dataset. C-MAPSS provides run-to-failure trajectories for multiple fleets of simulated turbofan engines operating under varying conditions and fault modes. The primary development subset is FD001, which contains 100 training engines and 100 test engines operating under a single condition with a single fault mode (High-Pressure Compressor degradation), amounting to approximately 20,631 labeled cycles.

Each cycle record contains 21 sensor measurements and 3 operational settings. An empirical variance analysis on the training data identified seven sensors (s1, s5, s6, s10, s16, s18, s19) as having zero or near-zero variance; these were dropped, leaving 14 informative sensor channels (s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21) used as model inputs.

2.2 Data Preprocessing

Raw sensor data is processed through the following pipeline:

- Min-max normalization is applied per sensor channel using statistics computed on the training set.
- The target RUL is clipped at a maximum value of 125 cycles, following the piece-wise linear degradation assumption commonly adopted in the C-MAPSS literature. This reflects the observation that healthy engines do not degrade meaningfully until a late stage of their life.
- Sliding windows of 30 consecutive cycles are extracted per engine unit, producing input tensors of shape (30 × 14) aligned to per-cycle RUL labels.
- The train/validation split is performed at the engine-unit level (80/20) rather than at the window level, to prevent temporal leakage between the two sets.

2.3 Model Architecture

APEX employs a hybrid Convolutional Neural Network – Bidirectional LSTM (CNN-BiLSTM) architecture for RUL regression. The convolutional front end extracts local degradation patterns across short temporal windows, while the bidirectional recurrent layers capture long-range degradation dynamics. The architecture consists of:

- Two stacked 1-D convolutional blocks (kernels of size 5 and 3, with 64 and 128 filters respectively), each followed by batch normalization and ReLU activation.
- A Bidirectional LSTM layer with 64 hidden units in each direction, producing a 128-dimensional temporal representation.
- A fully-connected head comprising two dense layers (64 units with ReLU, followed by a single-unit linear output) that regresses the RUL value for the current window.
- Dropout layers ($p = 0.3$) inserted after the convolutional and recurrent blocks, kept active at inference time to support Monte Carlo (MC) Dropout uncertainty estimation.

The choice of CNN-BiLSTM over a pure Transformer-based model was deliberate. While published state-of-the-art Transformers achieve slightly lower RMSE on FD001, they do so at roughly four times the inference latency. For a real-time multi-machine dashboard, predictable sub-15-millisecond inference is more valuable than a marginal improvement in point-estimate accuracy.

2.4 Uncertainty Quantification

Deterministic RUL predictions are insufficient for maintenance decision-making; operators require a measure of confidence. APEX uses Monte Carlo Dropout: at inference time, dropout layers remain active and 50 stochastic forward passes are performed per input window. The mean of the 50 predictions is reported as the point estimate, while the standard deviation provides a calibrated uncertainty band. These outputs are then mapped to urgency tiers (Healthy, Watch, Warning, Critical) using a rule-based classifier that combines predicted RUL, its uncertainty, and the slope of recent RUL estimates.

2.5 Plain-Language Advisory Generation

Raw numerical outputs are translated into maintenance advisories through a rule-based template engine driven by (a) the predicted urgency tier, (b) the dominant anomalous sensor channels identified via z-score analysis against healthy-operation baselines, and (c) a lookup of similar historical failure fingerprints from the training fleet. The result is a concise natural-language recommendation such as: "Replace high-pressure compressor bearing within the next shift; pattern matches failure signature of Engine_023 at cycle 187."

3. System Architecture

3.1 Architectural Overview

APEX follows a layered, modular architecture designed for low-latency real-time operation on commodity hardware. The high-level data flow is:

```
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ Sensor Source    │   │ Preprocessing    │   │ CNN-BiLSTM       │
│ (C-MAPSS replay  │─▶│ Normalize + 30-  │─▶│ + MC Dropout     │
│  or live IIoT)   │   │ cycle windows    │   │ (50 passes)      │
└──────────────────┘   └──────────────────┘   └──────────────────┘
                                                       │
                                                       ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ React Dashboard  │◀─│ WebSocket        │◀─│ FastAPI          │
│ (TypeScript,     │   │ /stream          │   │ Inference +      │
│ Tailwind,        │   │ (1 Hz frames)    │   │ Advisory Engine  │
│ Recharts)        │   │                  │   │                  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

Figure 1. High-level system architecture and data flow of APEX.

3.2 Component Description

- **Sensor Simulator:** A Python asyncio service replays C-MAPSS engine trajectories at configurable rates (default 2 Hz) to emulate live telemetry. In a production deployment, this component is replaced by an OPC-UA, MQTT, or Modbus adapter for real IIoT sources.
- **Inference Backend:** A FastAPI application exposes REST endpoints (/ingest, /predict/{id}, /health) and a WebSocket (/stream) that broadcasts prediction frames to all connected clients. Predictions are computed by loading the trained PyTorch checkpoint and executing the MC Dropout prediction loop in-process.
- **Advisory Engine:** A rule-based post-processor that consumes the raw model outputs and produces the structured advisory message, urgency tier, dominant-sensor diagnosis, and a counterfactual cost estimate.
- **Frontend Dashboard:** A React + TypeScript application using Tailwind CSS, Zustand for state management, and Recharts for visualization. It renders a fleet overview grid, per-machine detail views with live RUL charts and confidence bands, a ranked maintenance queue, and panels for six differentiator features (see Section 4.2).

4. Implementation Details

4.1 Technology Stack

Layer	Technology
Machine Learning	PyTorch 2.x, NumPy, Pandas, scikit-learn, XGBoost (baseline)
Backend	Python 3.11, FastAPI, Uvicorn, WebSockets, Pydantic
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Zustand, Recharts
Data	NASA C-MAPSS FD001 (100 train + 100 test engines, ~20,631 cycles)
Dev & Deployment	Git, GitHub, Docker (local), designed for deployment on Vercel (frontend) and Hugging Face Spaces (backend)
Notifications	Resend API (email alerts), Web Speech API (voice alerts)

Table 1. Technology stack used in the APEX implementation.

4.2 Key Implementation Features

Beyond the core RUL prediction pipeline, six differentiator features were implemented to extend the product from a prediction tool to a full maintenance advisor:

5. Fast-forward replay with scrub bar — compresses an engine's lifecycle into seconds and allows users to scrub to any point in the degradation timeline.
6. Failure counterfactual — on any Critical transition, the dashboard computes and displays the estimated rupee savings of acting on the APEX advisory versus running the machine to failure.
7. Rule-based AI insight panel — a plain-language explanation of which component is most likely failing, derived from per-sensor z-score deviations against healthy baselines.
8. Similar-failures retrieval — a nearest-neighbor lookup of the current sensor fingerprint against a library of historical failure signatures from the training fleet.
9. Real email alerts — automated email notifications triggered on Critical transitions, sent via the Resend API.
10. Voice alerts and keyboard shortcuts — spoken announcements of critical events using the Web Speech API, plus hotkeys for rapid navigation (1–5 select machines, space pauses simulation, e triggers email alert, ? opens a help overlay).

4.3 Training Configuration

Model training uses Huber loss ($\delta = 1.0$), the Adam optimizer with an initial learning rate of 1×10^{-3} and cosine-annealing schedule, a batch size of 256, and early stopping on validation RMSE with a patience of 7 epochs. All random seeds are fixed at 42 for full reproducibility.

4.4 Real-Time Serving

Predictions are streamed from the backend to the dashboard over a persistent WebSocket connection at 1 Hz per monitored machine. Each frame carries the current sensor reading, the point RUL estimate, its uncertainty band, the computed urgency tier, the advisory text, and a latency stamp that allows the frontend to display and monitor end-to-end performance in real time.

5. Results and Outputs

5.1 Model Performance

Table 2 summarizes the performance of APEX's CNN-BiLSTM model against an XGBoost baseline and a reported state-of-the-art Transformer benchmark on the C-MAPSS FD001 test set. Four metrics are reported: Root Mean Squared Error (RMSE), the official NASA asymmetric scoring function (which penalizes late predictions more heavily than early ones), the F1-score for a 30-cycle binary failure-horizon classification, and per-prediction inference latency.

Model	RMSE ↓	NASA Score ↓	F1 (30-cycle) ↑	Latency
XGBoost baseline	19.8	612	0.74	0.8 ms
CNN-BiLSTM (ours)	**14.2**	**328**	**0.89**	**11 ms**
SOTA Transformer (published)	11.3	192	—	45 ms

Table 2. Performance comparison on NASA C-MAPSS FD001 test set.

The proposed CNN-BiLSTM reduces RMSE by 28% (19.8 → 14.2) and NASA Score by 46% (612 → 328) relative to the XGBoost baseline, while lifting F1 for the 30-cycle failure-classification task from 0.74 to 0.89. Although the published Transformer SOTA achieves a somewhat lower RMSE, it does so at roughly four times the per-prediction latency — a trade-off that is unfavorable for the real-time multi-machine monitoring scenario APEX is built for.

5.2 System Latency

End-to-end latency was measured from the moment a sensor frame is emitted by the simulator to the moment the corresponding visual update is committed to the React DOM. The observed values are 0.4 seconds at the 50th percentile and 1.1 seconds at the 99th percentile, against a hackathon requirement of under 3 seconds. The system therefore operates with roughly 2.7× headroom at the worst observed percentile.

5.3 Dashboard Outputs

The deployed dashboard presents the following views, illustrated by screenshots in the accompanying demonstration artifacts:

- Fleet Overview: a grid of machine cards, each showing the current urgency tier color, the latest RUL prediction, its uncertainty band, and a sparkline of recent readings.
- Machine Detail: a per-machine page with a live time-series chart of predicted RUL, a 95% confidence band derived from MC Dropout, sensor anomaly indicators, and the full plain-language advisory.
- Maintenance Queue: a ranked list of all monitored machines sorted by urgency, supporting one-click acknowledgment of advisories.
- Latency Pill: a persistent HUD element displaying current p50/p99 dashboard latency for operator transparency.

5.4 Illustrative Advisory

> *"Engine_047 — CRITICAL. Predicted Remaining Useful Life: 8 cycles (±3 cycles, 95% CI). Failure probability within the next 30 cycles: 82%. Diagnosis: elevated vibration and exhaust-gas temperature consistent with High-Pressure Compressor bearing wear; pattern matches failure signature of Engine_023 at cycle 187. Recommended action: replace HPC bearing within the next shift. Estimated savings versus run-to-failure: ₹39,000."*

6. Conclusion and Future Scope

6.1 Conclusion

This paper presented APEX, an end-to-end AI-driven predictive maintenance advisor designed for the cost and deployment realities of SME manufacturing. The system ingests multivariate sensor telemetry, predicts Remaining Useful Life using a CNN-BiLSTM network with Monte Carlo Dropout uncertainty estimation, and delivers ranked plain-language maintenance advisories through a real-time web dashboard. On the NASA C-MAPSS FD001 benchmark, the proposed model achieves an RMSE of 14.2 cycles and an F1-score of 0.89 — substantially outperforming a traditional XGBoost baseline — while maintaining end-to-end dashboard latency well under the real-time threshold. APEX demonstrates that the gap between research-grade predictive maintenance and practical SME deployment can be closed using only open-source tooling and public benchmark data.

6.2 Future Scope

Several directions are identified for future work:

11. Generalization to additional C-MAPSS subsets (FD002, FD003, FD004) and to real industrial pump, motor, and compressor datasets, to validate robustness across operating conditions and fault modes.
12. Fine-tuning pipeline on customer-specific telemetry, allowing rapid adaptation of the base model to the specific equipment of a deploying SME.
13. Edge deployment via ONNX runtime in the browser and on low-cost hardware such as the Raspberry Pi, removing the dependence on a dedicated server and enabling fully on-premises operation.
14. Integration with industry-standard protocols (OPC-UA, MQTT, Modbus) to replace the simulator with real live sensor feeds.
15. A richer advisory engine powered by a compact large language model, producing advisories that adapt to the specific vocabulary and maintenance procedures of each deploying organization.
16. Multi-tenant SaaS deployment with authentication, role-based access control, and historical analytics for maintenance program optimization.

References

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. Int. Conf. on Prognostics and Health Management (PHM), 2008, pp. 1–9.

[2] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proc. Int. Conf. on Prognostics and Health Management, 2008, pp. 1–6.

[3] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in Proc. Int. Conf. on Machine Learning (ICML), 2016, pp. 1050–1059.

[4] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in IEEE Int. Conf. on Prognostics and Health Management (ICPHM), 2017, pp. 88–95.